VAE Complete Exam Notes — NPTEL Week 3 (Lec 19–24)

Course: Fundamentals of Generative AI and Large Language Models: Theory and Practice. Covers KL divergence → information theory → VAE encoder → decoder and ELBO → reparameterization trick → full numerical example. Includes worked numericals, a formula sheet and MCQs with answers.

0. Roadmap: which lecture covers what

The PDFs are numbered in reverse of teaching order. Here is the order in which the material builds:

- **Lec 19:** Intuition behind KL divergence (forward vs reverse KL, mode-covering vs mode-seeking)
- **Lec 20:** Self-information, entropy, cross-entropy, and the derivation of KL divergence
- **Lec 21:** Motivation for VAEs, the setup, and the **encoder**
- **Lec 22:** The **decoder**, the **ELBO**, and the loss function
- **Lec 23:** The **reparameterization trick**
- **Lec 24:** A complete **numerical forward pass** (encoder → sampling → decoder → loss)

There is no hands-on in Week 3; implementation is in Week 4.

The whole story in six lines

1. An autoencoder learns to *reconstruct*. A VAE is built to *generate* new data.
2. We want a model $p_\theta(x)$ that is close to the unknown true data distribution $p_{data}(x)$.
3. Directly computing $p_\theta(x)$ needs an integral over all latent z, which is intractable.
4. So the encoder learns an approximate posterior $q_\varphi(z|x) = N(\mu, \sigma^2)$, and we maximize a computable lower bound (the ELBO) instead of $\log p(x)$.
5. ELBO = reconstruction term − $KL(q(z|x) \| p(z))$. Flip the sign to get the loss.
6. Sampling z blocks gradients, so we write $z = \mu + \sigma \odot \varepsilon$ with $\varepsilon \sim N(0, I)$. That is the reparameterization trick.

---

1. Lec 19: Intuition behind KL divergence

1.1 Why we need it

Two points in 2D can be compared with Euclidean distance $\sqrt{((x_2-x_1)^2 + (y_2-y_1)^2)}$. But how do you measure how different two **probability distributions** P(x) and Q(x) are? The answer is the **Kullback–Leibler (KL) divergence**. It works for both discrete and continuous distributions.

1.2 What it means

KL divergence measures how much a distribution Q(x) differs from a reference distribution P(x).

- **Identical distributions** (P = Q): complete overlap, KL = 0.
- **Slightly different** (nearby means, good overlap): KL is small.
- **Very different** (widely separated means, little overlap): KL is large.

Key properties (high-yield for exams)

- KL ⩾ 0 always, and KL = 0 if and only if P = Q.
- KL is **not symmetric**: $D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$ in general.
- Because of the asymmetry (and no triangle inequality) it is a *divergence*, **not a true distance/metric**.

1.3 Forward KL: $D_{KL}(P \| Q)$

Discrete form: $D_{KL}(P\|Q) = \Sigma_x P(x) \log [P(x)/Q(x)]$

Continuous form: $D_{KL}(P\|Q) = \int P(x) \log [P(x)/Q(x)] dx$

- The expectation is taken under **P (the true distribution)**.
- If P(x) > 0 but Q(x) = 0 somewhere, then $\log(P/0) \to \infty$, so the KL is **infinite**. Forward KL heavily penalizes Q for **missing regions where P has mass**.
- Result: Q is pushed to **spread out and cover the whole support of P**. This is **mode-covering** behaviour.
- If P is bimodal and Q is a single Gaussian, Q becomes broad and sits across both modes (and also across the low-density gap in between).

1.4 Reverse KL: $D_{KL}(Q \| P)$

Discrete form: $D_{KL}(Q\|P) = \Sigma_x\, Q(x) \log [Q(x)/P(x)]$

Continuous form: $D_{KL}(Q\|P) = \int Q(x) \log [Q(x)/P(x)]\, dx$

- The expectation is taken under **Q (the model)**.
- If $Q(x) > 0$ where $P(x) \approx 0$, the ratio Q/P blows up, so the penalty is very large. Reverse KL penalizes Q for **putting probability where P has little or no mass**.
- Result: Q concentrates on **one high-density mode** of P and may ignore the others. This is **mode-seeking** behaviour.

1.5 Mode-covering vs mode-seeking: the examples from the lecture

Height example. Real data has short people around 160 cm and tall people around 190 cm, with almost nobody near 175 cm. P(x) has two modes (Cluster A and Cluster B). A single Gaussian Q is fitted.

	Forward KL, D(P‖Q)	Reverse KL, D(Q‖P)
Behaviour	Mode-covering	Mode-seeking
What Q does	Becomes broad, covers both clusters	Locks onto 160 cm **or** 190 cm
Advantage	Both groups represented	Samples from the chosen mode are realistic
Limitation	Also puts mass near 175 cm (the low-density gap)	The other valid mode is missed

Cats and dogs example. Dataset has cat and dog images; the model must generate animals.

- **Forward KL (limited capacity):** tries to cover both, so it produces ambiguous, unrealistic in-between images that look partly like a cat and partly like a dog.
- **Reverse KL (limited capacity):** learns only cats or only dogs and produces sharp, clear images, but never generates the other class.

One-line memory hook: Forward KL = *cover everything, even if blurry*. Reverse KL = *be sharp, even if you miss modes*.

1.6 Role in generative modelling

Real samples come from an unknown P(x). A generative model learns $Q_\theta(x)$ with the goal $Q_\theta(x) \approx P(x)$. KL divergence quantifies the mismatch. (In the lecture's two-feature dataset example, Feature 1 is roughly Gaussian and Feature 2 is bimodal, so fitting a single Gaussian shows the two behaviours clearly.)

---

2. Lec 20: Self-information, entropy, cross-entropy, KL derivation

Running example: tomorrow's weather $X \in \{\text{Sunny, Rainy}\}$, with a model predicting P(Sunny) = 0.875 and P(Rainy) = 0.125.

2.1 Self-information (surprise of ONE outcome)

$I(x) = \log_2 [1/P(x)] = -\log_2 P(x)$

- P(x) large → I(x) small (little surprise).
- P(x) small → I(x) large (big surprise). That is why the formula has 1/P(x).
- With $\log_2$ the unit is **bits** (with ln it would be nats).
- **Why log?** It turns multiplication of independent probabilities into addition of information: log(ab) = log a + log b. So I(A and B) = I(A) + I(B) for independent events.

Numbers:

- $I(\text{Sunny}) = -\log_2(0.875) \approx$ **0.193 bits** (not surprising)
- $I(\text{Rainy}) = -\log_2(0.125) =$ **3 bits** (surprising, since $0.125 = 2^{-3}$)

2.2 Entropy (average surprise of the WHOLE distribution)

Self-information describes one observed outcome. Entropy gives one number for the uncertainty of the entire distribution, by averaging surprise weighted by probability:

$H(P) = \Sigma_x P(x)\, I(x) = -\Sigma_x P(x) \log_2 P(x) = \Sigma_x P(x) \log_2 [1/P(x)]$

Worked example (the lecture's):

$H(P) = 0.875 \cdot \log_2(1/0.875) + 0.125 \cdot \log_2(1/0.125) = 0.875 \times 0.193 + 0.125 \times 3 = 0.169 + 0.375 =$ **0.544 bits**

A fairly confident model (one outcome much more likely) gives **low** entropy.

Extremes to remember:

Distribution	Entropy	Meaning
[0.5, 0.5]	H = 1 bit (maximum for 2 outcomes)	Most unpredictable
[1, 0] or [0, 1]	H = 0	No uncertainty, fully predictable
Uniform over n outcomes	H = $\log_2 n$ (the maximum)	Maximum uncertainty

Convention: $0 \cdot \log 0 = 0$.

2.3 Cross-entropy (how surprised is the MODEL about the TRUTH)

For one outcome x the model Q assigns surprise $I_Q(x) = -\log_2 Q(x)$. Weight it by the **true** probability P(x):

$H(P, Q) = \Sigma_x P(x)\, I_Q(x) = -\Sigma_x P(x) \log_2 Q(x)$

- P = true/target distribution, Q = model's predicted distribution.
- Small when the model gives high probability to the correct outcome; large when it gives low probability to the correct outcome.
- When P is one-hot (true class has probability 1), cross-entropy collapses to **−log Q(correct class)**. This is exactly the classification loss.

Worked examples (true outcome = Sunny, so P = [1, 0] = [Sunny, Rainy]):

- Good model, Q = [0.875, 0.125]: $H(P,Q) = -[1 \cdot \log_2 0.875 + 0 \cdot \log_2 0.125] = -\log_2(0.875) \approx$ **0.193** (small loss)
- Bad model, Q = [0.125, 0.875]: $H(P,Q) = -\log_2(0.125) =$ **3** (large loss, because only 0.125 was assigned to the correct class)

2.4 Derivation of KL divergence (learn this; it is a classic exam question)

KL is the **extra** cost of using Q when the truth is P:

$D_{KL}(P \| Q) = H(P, Q) - H(P)$

Substitute the definitions:

1. $D_{KL} = [-\Sigma\, P \log_2 Q] - [-\Sigma\, P \log_2 P]$
2. $= -\Sigma\, P \log_2 Q + \Sigma\, P \log_2 P$

3. Rearrange: = Σ P $\log_2$ P − Σ P $\log_2$ Q
4. Take P(x) common: = Σ P(x) [$\log_2$ P(x) − $\log_2$ Q(x)]
5. Use log a − log b = log(a/b):

$D_{KL}(P\|Q) = \Sigma_x P(x) \log_2 [P(x)/Q(x)]$

Interpretation: cross-entropy = entropy + KL, i.e. H(P,Q) = H(P) + D_{KL}(P‖Q). Entropy is the unavoidable uncertainty; KL is the *extra* bits you pay for using the wrong distribution Q. Since H(P) is fixed by the data, **minimizing cross-entropy over Q is the same as minimizing KL**.

2.5 Extra numerical: KL is asymmetric (practice)

Let P = [0.875, 0.125] and Q = [0.5, 0.5].

Forward KL: D(P‖Q) = 0.875·$\log_2$(0.875/0.5) + 0.125·$\log_2$(0.125/0.5) = 0.875·$\log_2$(1.75) + 0.125·$\log_2$(0.25) = 0.875(0.807) + 0.125(−2) = 0.706 − 0.25 = **0.456 bits**

(Check with the identity: H(P,Q) = −Σ P $\log_2$ Q = 1 bit; H(P) = 0.544; 1 − 0.544 = 0.456 ✓)

Reverse KL: D(Q‖P) = 0.5·$\log_2$(0.5/0.875) + 0.5·$\log_2$(0.5/0.125) = 0.5(−0.807) + 0.5(2) = −0.404 + 1 = **0.596 bits**

So D(P‖Q) = 0.456 ≠ D(Q‖P) = 0.596. **KL is not symmetric.**

---

3. Lec 21: VAE motivation, setup and the encoder

3.1 Motivation

- An autoencoder learned **reconstruction**.
- A VAE is introduced because we want **generation** (generative modelling).

3.2 The generative-modelling goal

Given a dataset X = {$x^{(1)}$, $x^{(2)}$, …, $x^{(N)}$} of N samples:

- The data come from an unknown distribution $p_{data}(x)$. We do **not** know its form (Gaussian, Bernoulli, Poisson …) or its parameters. We only have a **finite sample** from it.
- VAE is **unsupervised**: class labels are not used. Classes still exist implicitly (e.g. in Iris, Setosa samples follow one feature pattern, Virginica another, each with its own class-conditional distribution p(x|c)), so the model must discover the underlying pattern by

itself.

- **Goal:** learn an approximate distribution $p_\theta(x) \approx p_{data}(x)$, where θ = the network's weights and biases.
- At the start of training $p_\theta(x)$ can be very different from $p_{data}(x)$. As θ is updated it gets closer.
- After learning, **generate new data by sampling: $x_{new} \sim p_\theta(x)$**.

3.3 Two spaces, two distributions, two mappings

- **Data space:** x lives here, with unknown distribution $p_{data}(x)$.
- **Latent space:** z lives here, with distribution p(z).
- x and z have different dimensionality and structure, so we need **probabilistic mappings** between the spaces.

Symbol	Name	Role
$p_{data}(x)$	Data distribution	Unknown; what we want to model
$p(z)$	**Prior**	Chosen by us: N(0, I)
$p(z\|x)$	**True posterior**	Which z produced x? Intractable
$p(x\|z) = p_\theta(x\|z)$	**Likelihood** (decoder)	Probability of data given latent
$q_\varphi(z\|x)$	**Approximate posterior** (encoder)	Learned stand-in for $p(z\|x)$
$p(x)$	**Evidence**	$\int p(x\|z)\, p(z)\, dz$

- **Encoder** = approximate posterior $q_\varphi(z|x)$: maps an input x to a *distribution* over z.
- **Decoder** = likelihood $p_\theta(x|z)$: maps z back to data space.
- φ = encoder parameters, θ = decoder parameters.

3.4 Why the true posterior is hard

Bayes' rule:

$$p(z|x) = p(x|z) \cdot p(z) / p(x)$$

The denominator is the evidence:

$$p(x) = \int p(x|z)\, p(z)\, dz$$

Computing it needs **integration over all possible values of the latent variable z**. For a neural-network decoder this integral is **intractable**. So in a VAE we do not compute p(z|x); we approximate it with $q_\varphi(z|x) \approx p(z|x)$, and the encoder network learns q_φ.

3.5 The latent variable is a vector

$z = [z_1, z_2, \ldots, z_d]$; each z_i is a latent dimension. Example: input with 100 features → Encoder → 25, so z is a **25-dimensional vector**. The encoder learns a meaningful **lower-dimensional** representation.

3.6 Choosing the prior

We do not know the true structure of the latent space, so we **choose** the prior to be a standard normal: $\mathbf{p(z_i) = N(0, 1)}$, i.e. $p(z) = N(0, I)$.

Why Gaussian?

- Simple and mathematically convenient.
- Sampling is easy.
- Encourages a well-structured, continuous latent space.

3.7 What the encoder outputs

- We *assume* $q_\varphi(z|x)$ is Gaussian: $q_\varphi(z|x) = N(\mu_\varphi(x), \sigma^2_\varphi(x))$. A Gaussian needs only **two parameters**, so the encoder only has to output $\mu_\varphi(x)$ and $\sigma^2_\varphi(x)$. That makes training easier.
- Choosing both $q_\varphi(z|x)$ and $p(z)$ as Gaussians also makes them **easy to compare**.
- **Who ensures q stays close to the Gaussian prior?** The **KL-divergence term** $D_{KL}(q_\varphi(z|x) \| p(z))$ in the loss.
- Once μ and σ^2 are known, sample $z \sim N(\mu, \sigma^2)$.

Practical detail: the encoder actually outputs **$\boldsymbol{\mu}$** and **$\boldsymbol{\log \sigma^2}$** (log-variance). Then $\sigma^2 = \exp(\log \sigma^2)$, which is always positive, so the network's raw output can be any real number. Neither μ nor $\log \sigma^2$ has an activation function (they can be positive, negative or zero).

3.8 The 5-dimensional latent example from the lecture

Latent dimension = 5. Encoder outputs:

- $\mu = [0.5, -0.2, 1.0, 0.3, -0.5]$
- $\log \sigma^2 = [-1.386, -2.303, -0.693, -1.833, -1.386]$

Convert with $\sigma^2 = \exp(\log \sigma^2)$:

- e^(−1.386) = 0.25, e^(−2.303) = 0.10, e^(−0.693) = 0.50, e^(−1.833) = 0.16, e^(−1.386) = 0.25
- σ^2 = [0.25, 0.10, 0.50, 0.16, 0.25]

So the five latent variables follow (second number is the **variance**):

- $z_1 \sim N(0.5, 0.25)$, $z_2 \sim N(-0.2, 0.10)$, $z_3 \sim N(1.0, 0.50)$, $z_4 \sim N(0.3, 0.16)$, $z_5 \sim N(-0.5, 0.25)$

Sample one value from each. Suppose we get z = [0.65, −0.10, 0.80, 0.42, −0.70]. That vector goes to the decoder.

Exam tip: the standard deviation is $\sigma = \sqrt{\sigma^2}$, so for z_1 the std is 0.5, not 0.25.

---

4. Lec 22: Decoder, ELBO and the loss function

4.1 What the decoder does

Once the encoder gives $\mu_\varphi(x)$ and $\sigma^2_\varphi(x)$, we sample $z \sim N(\mu_\varphi(x), \sigma^2_\varphi(x))$ and pass z to the decoder. The decoder is a neural network that takes z and outputs the **parameters of the likelihood distribution $p_\theta(x|z)$**. Which parameters depends on the data type:

Data type	Decoder outputs	Distribution	Generated output
Continuous / real-valued	$\mu_\theta(z)$ and $\sigma^2_\theta(z)$	$p_\theta(x\|z) = N(\mu_\theta(z), \sigma^2_\theta(z))$	$\hat{x} \sim N(\mu_\theta(z), \sigma^2_\theta(z))$
Binary (or pixels in [0,1])	$\pi_\theta(z)$, a probability per feature/pixel	$p_\theta(x\|z) = \text{Bernoulli}(\pi_\theta(z))$	$\hat{x} \sim \text{Bernoulli}(\pi_\theta(z))$

So the generated $\hat{x}$ is *randomly drawn* from that distribution.

4.2 One input x has many possible z (lecture example)

Input x = [0.80, 0.10, 0.60, 0.30, 0.90]. Encoder gives $\mu(x)$ = [0.50, −0.20] and $\sigma^2(x)$ = [0.04, 0.09]. So $z_1 \sim N(0.50, 0.04)$ and $z_2 \sim N(-0.20, 0.09)$.

Three samples from the **same** distribution $q_\varphi(z|x)$:

- z^1 = [0.56, −0.11] → Decoder → $\hat{x}^1$ = [0.79, 0.12, 0.58, 0.32, 0.88]
- z^2 = [0.43, −0.32] → Decoder → $\hat{x}^2$ = [0.76, 0.14, 0.61, 0.28, 0.86]

- $z^3 = [0.61, -0.24] \rightarrow$ Decoder $\rightarrow \hat{x}^3 = [0.82, 0.09, 0.57, 0.34, 0.91]$

The reconstructions differ slightly (different z) but all stay close to x. **Takeaway:** one observed x corresponds to **infinitely many** possible z. The reverse question is: *given a real training sample x, which latent z produces it through the decoder?* That is the posterior p(z|x) again.

4.3 Why we cannot just use Bayes' rule here

The prior is p(z) = N(0, I). Each candidate z contributes p(x|z)·p(z): the decoder's probability that this z generates x, times the prior probability of that z. Latent vectors near the origin have high prior density (e.g. z = [0.2, −0.1]); vectors far away have very low density (e.g. z = [4.0, −3.5]). To get p(x) you must sum these contributions over **all** z, which is the intractable integral $p(x) = \int p(x|z)p(z)dz$.

4.4 Evidence Lower Bound (ELBO): the idea

Suppose the training data are handwritten digits. After training the model should give **high probability to digit-like images and low probability to random noise**. Ideally we would maximize **log $p_\theta(x)$**. But computing $p_\theta(x)$ requires checking all latent z that could have generated x, so it is not directly possible. VAE therefore maximizes a **computable lower bound**, the **Evidence Lower Bound (ELBO)**: ELBO $\leqslant \log p_\theta(x)$.

The ELBO:

$$\mathbf{L_{ELBO}(x) = E_{q_\varphi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\varphi(z|x) \,\|\, p(z))}$$

- **Term 1, reconstruction:** if z is sampled from the encoder, can the decoder reconstruct x with high probability?
- **Term 2, KL regularization:** is the encoder distribution $q_\varphi(z|x)$ close to the prior p(z)?

Why it is a lower bound (derivation, good for long-answer questions). Start with log p(x) and insert q:

1. $\log p(x) = E_q[\log p(x)]$ (p(x) does not depend on z)
2. $= E_q[\log p(x,z)/p(z|x)]$ (since $p(x,z) = p(z|x)p(x)$)
3. $= E_q[\log (p(x,z)/q(z|x)) \cdot (q(z|x)/p(z|x))]$
4. $= E_q[\log p(x,z)/q(z|x)] + E_q[\log q(z|x)/p(z|x)]$
5. The second term is $D_{KL}(q(z|x) \,\|\, p(z|x)) \geqslant 0$. The first term is the ELBO.

So **log p(x) = ELBO + D_KL(q(z|x) ‖ p(z|x)) ⩾ ELBO**. Maximizing the ELBO pushes log p(x) up and also squeezes q toward the true posterior. Finally, expanding log p(x,z) = log p(x|z) + log p(z) gives ELBO = E_q[log p(x|z)] − D_KL(q(z|x) ‖ p(z)).

4.5 From ELBO to the VAE loss

The first term is a log-likelihood; we want the model to assign **high probability to observed data**, i.e. *maximize* it. Neural networks are trained by **minimizing** a loss, and max(f) ⇔ min(−f). So:

L_VAE(x) = −E_{q_φ(z|x)}[log p_θ(x|z)] + D_KL(q_φ(z|x) ‖ p(z))

L_VAE = Reconstruction Loss + KL Divergence Loss

4.6 The reconstruction term in practice

- **Continuous data → Gaussian likelihood → MSE.** With a fixed-variance Gaussian, −log p(x|z) reduces (up to constants) to the squared error ‖x − μ_x‖². The lecture uses Reconstruction Loss = (1/n) Σ $(x_i - \mu_{x,i})^2$.
- **Binary / [0,1] data → Bernoulli likelihood → Binary Cross-Entropy (BCE).**

The role of this term: make x̂ close to x.

4.7 The KL term in closed form

Both q_φ(z|x) = N(μ, σ²) and p(z) = N(0, 1) are Gaussian, so the KL has a **closed-form** expression (no sampling needed):

L_KL = KL(N(μ, σ²) ‖ N(0,1)) = ½ Σ_j ($\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1$)

(log here is the natural log; sum over the latent dimensions j.)

Where it comes from (short derivation, 1-D): KL = E_q[log q(z) − log p(z)]. With log q(z) = −½log(2πσ²) − (z−μ)²/(2σ²) and log p(z) = −½log(2π) − z²/2, and using E_q[(z−μ)²] = σ² and E_q[z²] = μ² + σ²: KL = −½logσ² − ½ + ½(μ² + σ²) = ½(μ² + σ² − logσ² − 1).

Sanity checks (very common exam questions):

- μ = 0, σ² = 1: KL = ½(0 + 1 − 0 − 1) = **0** (q already equals the prior).
- μ = 2, σ² = 1: KL = ½(4 + 1 − 0 − 1) = **2**.
- μ = 0, σ² = e: KL = ½(0 + 2.718 − 1 − 1) = **0.359**.
- KL penalizes **both** a mean far from 0 (the μ² term) and a variance away from 1 (the σ² − logσ² − 1 term, which is zero only at σ² = 1).

Role of the two terms (tension): reconstruction wants q to be sharp and specific to each x (small σ, informative μ); the KL term wants q to be like N(0, 1) for every x. Balancing them gives a smooth, continuous latent space you can sample from. KL alone would collapse every x to the prior (output ignores input); reconstruction alone would behave like a plain autoencoder with a gappy latent space.

Direction of the KL used: $D_{KL}(q \| p(z))$ is a KL with q first, so it is the **reverse-KL direction** from Lec 19.

4.8 Remaining problem

$z \sim q_\varphi(z|x)$ is a random sampling step, and **randomness breaks gradient flow, making backpropagation impossible**. How do we compute gradients when part of the network involves sampling? This motivates the reparameterization trick.

---

5. Lec 23: The reparameterization trick

5.1 Decoder side is easy (standard backprop)

The decoder is an ordinary MLP with parameters $\theta = \{W_1, b_1, W_2, b_2\}$:

- **Layer 1:** $h_1 = \text{ReLU}(W_1 z + b_1)$
- **Output layer:** $\hat{x} = f(W_2 h_1 + b_2)$, where f = **sigmoid** for binary data or [0,1]-normalized pixels, and **linear** for continuous-valued data.
- Compute Loss($\hat{x}$, x), then update θ with the chain rule:
  - $\partial L/\partial W_2 = \partial L/\partial \hat{x} \cdot \partial \hat{x}/\partial W_2$, $\partial L/\partial b_2 = \partial L/\partial \hat{x} \cdot \partial \hat{x}/\partial b_2$
  - $\partial L/\partial W_1 = \partial L/\partial \hat{x} \cdot \partial \hat{x}/\partial h_1 \cdot \partial h_1/\partial W_1$, $\partial L/\partial b_1 = \partial L/\partial \hat{x} \cdot \partial \hat{x}/\partial h_1 \cdot \partial h_1/\partial b_1$

z is treated as a numeric input and the decoder is differentiable w.r.t. it, so gradients also reach z: $\partial L/\partial z = \partial L/\partial \hat{x} \cdot \partial \hat{x}/\partial z$. **The decoder side is not the main difficulty.**

5.2 The real problem: the encoder

To update the encoder we need $\partial L/\partial \mu_\varphi$ and $\partial L/\partial \sigma_\varphi$. By the chain rule:

- $\partial L/\partial \mu_\varphi = \partial L/\partial z \cdot \partial z/\partial \mu_\varphi$
- $\partial L/\partial \sigma_\varphi = \partial L/\partial z \cdot \partial z/\partial \sigma_\varphi$

But $z \sim N(\mu_\varphi, \sigma^2_\varphi)$ is obtained by **random sampling**, so z is not written as a differentiable function of μ_φ and σ_φ, and $\partial z/\partial\mu$, $\partial z/\partial\sigma$ do not exist. **Direct backpropagation through sampling is not possible.**

5.3 The trick

Instead of sampling z directly, **separate the randomness from the parameters**:

$z = \mu_\varphi(x) + \sigma_\varphi(x) \odot \varepsilon$, with $\varepsilon \sim N(0, I)$

- $\mu_\varphi(x)$, $\sigma_\varphi(x)$ come from the encoder (depend on x and φ).
- ε is **not learned**; it is drawn from a standard normal independent of μ, σ and φ. The network learns μ and σ, not ε.
- During backprop, ε is treated as a **constant**. So z becomes a **deterministic, differentiable function** of μ and σ (the randomness lives only in ε).
- ⊙ is element-wise multiplication.
- It is valid because if $\varepsilon \sim N(0,1)$ then $\mu + \sigma\varepsilon \sim N(\mu, \sigma^2)$. The distribution of z is unchanged.

Computational graph view: before, z was a *random node* $\sim q(z|x)$ with arrows from μ and σ, so gradients stop there. After, $z = \mu + \sigma \odot \varepsilon$ is a *deterministic node* with inputs μ, σ and an external random node $\varepsilon \sim N(0,1)$; gradients flow through μ and σ to the encoder.

5.4 The two partial derivatives

$z = \mu + \sigma \odot \varepsilon$

- **W.r.t. μ:** $\partial z/\partial\mu = \partial\mu/\partial\mu + \partial(\sigma\varepsilon)/\partial\mu = 1 + 0 =$ **1** ($\sigma\varepsilon$ does not contain μ).
- **W.r.t. σ:** $\partial z/\partial\sigma = \partial\mu/\partial\sigma + \partial(\sigma\varepsilon)/\partial\sigma = 0 + \varepsilon =$ **ε** (μ does not depend on σ; ε is a constant).

5.5 Full gradient path

Path: **$L \rightarrow \hat{x} \rightarrow z \rightarrow (\mu_\varphi(x), \sigma_\varphi(x)) \rightarrow \varphi$**

- $\partial L/\partial\mu_\varphi = \partial L/\partial\hat{x} \cdot \partial\hat{x}/\partial z \cdot \partial z/\partial\mu_\varphi = \partial L/\partial\hat{x} \cdot \partial\hat{x}/\partial z$ (since $\partial z/\partial\mu = 1$)
- $\partial L/\partial\sigma_\varphi = \partial L/\partial\hat{x} \cdot \partial\hat{x}/\partial z \cdot \varepsilon$ (since $\partial z/\partial\sigma = \varepsilon$)
- Then to the encoder parameters: $\partial L/\partial\varphi = (\partial L/\partial\mu_\varphi)(\partial\mu_\varphi/\partial\varphi) + (\partial L/\partial\sigma_\varphi)(\partial\sigma_\varphi/\partial\varphi)$

Conclusion: the reparameterization trick lets gradients flow from the loss all the way back to the encoder parameters, enabling **end-to-end training** of the VAE.

5.6 Mini numerical example

Using Lec 24's first latent unit: $\mu_1 = 0.043$, $\sigma_1 = 1.442$, and suppose $\varepsilon_1 = 0.5$. Then $z_1 = 0.043 + 1.442 \times 0.5 = 0.764$. Suppose the decoder gives $\partial L/\partial z_1 = 2.0$. Then $\partial L/\partial \mu_1 = 2.0 \times 1 = \mathbf{2.0}$ and $\partial L/\partial \sigma_1 = 2.0 \times 0.5 = \mathbf{1.0}$. If ε_1 had been -0.4 the σ-gradient would be $2.0 \times (-0.4) = -0.8$: the gradient to σ depends on which ε was drawn, the gradient to μ does not.

(Implementation note: networks usually output $\log \sigma^2$, so $\sigma = \exp(\tfrac{1}{2} \log \sigma^2)$ and the gradient flows one step further through that exp.)

---

6. Lec 24: Complete numerical example (full forward pass + loss)

This is the most **exam-likely** part. Architecture used: **4 inputs → 3 hidden (ReLU) → latent 2 (μ and log σ²) → sample z → 3 hidden (ReLU) → 4 outputs (linear)**.

Input: **x = [5.1, 3.5, 1.4, 0.2]** (an Iris-type sample).

Step 1: Encoder hidden layer

$h = x W_1 + b_1$ with $b_1 = 0.1$ and

W_1 (4×3) = [[0.1, −0.2, 0.3], [0.4, 0.5, −0.6], [−0.7, 0.8, 0.9], [0.2, −0.1, 0.4]]

(Each column of W_1 feeds one hidden neuron.)

- $h_1 = 5.1(0.1) + 3.5(0.4) + 1.4(-0.7) + 0.2(0.2) + 0.1 = 0.51 + 1.40 - 0.98 + 0.04 + 0.1 = \mathbf{1.07}$
- $h_2 = 5.1(-0.2) + 3.5(0.5) + 1.4(0.8) + 0.2(-0.1) + 0.1 = -1.02 + 1.75 + 1.12 - 0.02 + 0.1 = \mathbf{1.93}$
- $h_3 = 5.1(0.3) + 3.5(-0.6) + 1.4(0.9) + 0.2(0.4) + 0.1 = 1.53 - 2.10 + 1.26 + 0.08 + 0.1 = \mathbf{0.87}$

ReLU: $a_i = \max(0, h_i)$. All three are positive, so **a = [1.07, 1.93, 0.87]**.

Step 2: Mean vector μ (latent dimension = 2)

μ = a W_μ + b_μ, with W_μ (3×2) = [[0.5, 0.2], [−0.3, 0.4], [0.1, −0.5]] and b_μ = 0.

- $\mu_1 = 1.07(0.5) + 1.93(-0.3) + 0.87(0.1) = 0.535 - 0.579 + 0.087 = \mathbf{0.043}$
- $\mu_2 = 1.07(0.2) + 1.93(0.4) + 0.87(-0.5) = 0.214 + 0.772 - 0.435 = \mathbf{0.551}$

μ = [0.043, 0.551]

No activation is applied to the μ layer. The mean can be positive, negative or zero (e.g. μ = [−2.5, 0.8] is perfectly valid). Applying ReLU would force ReLU(−2.5) = 0 and wrongly forbid negative means.

Step 3: Log-variance vector

$\log \sigma^2 = a W_\sigma + b_\sigma$, with W_σ (3×2) = [[−0.2, 0.3], [0.4, −0.1], [0.2, 0.2]] and $b_\sigma = 0$.

- $\log \sigma_1^2 = 1.07(-0.2) + 1.93(0.4) + 0.87(0.2) = -0.214 + 0.772 + 0.174 =$ **0.732**
- $\log \sigma_2^2 = 1.07(0.3) + 1.93(-0.1) + 0.87(0.2) = 0.321 - 0.193 + 0.174 =$ **0.302**

$\log \sigma^2$ = [0.732, 0.302]. Again **no activation** on this layer.

Step 4: Convert to variance and standard deviation

$\sigma^2 = e^{\wedge}(\log \sigma^2)$, $\sigma = \sqrt{(\sigma^2)}$

- $\sigma_1^2 = e^{\wedge}0.732 =$ **2.079**, $\sigma_2^2 = e^{\wedge}0.302 =$ **1.353**
- $\sigma_1 = \sqrt{2.079} =$ **1.442**, $\sigma_2 = \sqrt{1.353} =$ **1.163**

So for this one input x the encoder defines **one** distribution $q_\varphi(z|x) = N(\mu = [0.043, 0.551], \sigma = [1.442, 1.163])$.

Step 5: Sampling from the latent space (reparameterization)

$z_1 = \mu_1 + \sigma_1\varepsilon_1$ and $z_2 = \mu_2 + \sigma_2\varepsilon_2$, with $\varepsilon \sim N(0, 1)$. **Different ε give different z from the same distribution.**

Sample	$\varepsilon = [\varepsilon_1, \varepsilon_2]$	z_1	z_2	z
1	[0.5, −0.2]	0.043 + 1.442(0.5) = 0.043 + 0.721 = 0.764	0.551 + 1.163(−0.2) = 0.551 − 0.233 = 0.318	[0.764, 0.318]
2	[0, 0]	0.043 + 0 = 0.043	0.551 + 0 = 0.551	[0.043, 0.551]
3	[0.6, −0.4]	0.043 + 1.442(0.6) = 0.043 + 0.865 = 0.908	0.551 + 1.163(−0.4) = 0.551 − 0.465 = 0.086	[0.908, 0.086]

Note: **ε = 0 gives z = μ** (the mean of the distribution). All three are samples from $q_\varphi(z|x)$. The lecture continues with **z = [0.764, 0.318]**.

Step 6: Decoder hidden layer

$h = z W_2 + b_2$, with decoder architecture z (2 units) → h (3 units), W_2 (2×3) = [[0.6, −0.4, 0.2], [0.3, 0.1, −0.5]], $b_2 = 0.1$.

- h_1 = 0.764(0.6) + 0.318(0.3) + 0.1 = 0.4584 + 0.0954 + 0.1 = **0.6538** → ReLU = 0.6538
- h_2 = 0.764(−0.4) + 0.318(0.1) + 0.1 = −0.3056 + 0.0318 + 0.1 = **−0.1738** → ReLU = **0** (negative, clipped)
- h_3 = 0.764(0.2) + 0.318(−0.5) + 0.1 = 0.1528 − 0.159 + 0.1 = **0.0938** → ReLU = 0.0938

a = [0.6538, 0, 0.0938]. Unlike the μ and log σ² layers, this is an ordinary hidden layer, so **ReLU is applied after computing h**.

Step 7: Decoder output layer

The activation on the output layer depends on the data type:

- **Continuous data → linear activation:** $\hat{x} = a W_3 + b_3$
- **Data normalized to [0,1] or binary → sigmoid:** $\hat{x} = \text{sigmoid}(a W_3 + b_3)$

Here the data are continuous, so **linear** is used. W_3 (3×4) = [[0.2, −0.1, 0.3, 0.4], [0.1, 0.2, −0.2, 0.1], [−0.3, 0.4, 0.1, −0.2]], b_3 = 0.

- $\hat{x}_1$ = 0.6538(0.2) + 0(0.1) + 0.0938(−0.3) = 0.13076 + 0 − 0.02814 = **0.10262**
- $\hat{x}_2$ = 0.6538(−0.1) + 0(0.2) + 0.0938(0.4) = −0.06538 + 0 + 0.03752 = **−0.02786**
- $\hat{x}_3$ = 0.6538(0.3) + 0(−0.2) + 0.0938(0.1) = 0.19614 + 0 + 0.00938 = **0.20552**
- $\hat{x}_4$ = 0.6538(0.4) + 0(0.1) + 0.0938(−0.2) = 0.26152 + 0 − 0.01876 = **0.24276**

$\hat{x}$ = μ_x = [0.103, −0.028, 0.206, 0.243] (rounded).

Theory vs practice: theoretically the reconstruction is *sampled* from p_θ(x|z); in practice the decoder output is treated as the **mean** of that distribution and used directly: $\hat{x} = \mu_x$.

Why is the reconstruction so bad (x = [5.1, 3.5, 1.4, 0.2] vs $\hat{x} \approx$ [0.10, −0.03, 0.21, 0.24])? Because the weights and biases are **randomly initialized**. After training, the parameters are optimized so that $\hat{x}$ becomes close to x.

Step 8: Reconstruction loss (MSE, continuous data)

Reconstruction Loss = $(1/4) \Sigma (x_i - \mu_{x,i})^2$

- $(5.1 - 0.103)^2 = (4.997)^2 = 24.9700$
- $(3.5 - (-0.028))^2 = (3.528)^2 = 12.4468$
- $(1.4 - 0.206)^2 = (1.194)^2 = 1.4256$
- $(0.2 - 0.243)^2 = (-0.043)^2 = 0.0018$

Sum = 38.8443, so Reconstruction Loss = 38.8443 / 4 = **9.7111**

Step 9: KL-divergence loss

$L_{KL} = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)$, using $\mu = [0.043, 0.551]$, $\log \sigma^2 = [0.732, 0.302]$, $\sigma^2 = [2.079, 1.353]$.

- $\mu_1^2 = 0.043^2 = 0.00185$, $\mu_2^2 = 0.551^2 = 0.3036$
- j = 1: 0.00185 + 2.079 − 0.732 − 1 = **0.34885**
- j = 2: 0.3036 + 1.353 − 0.302 − 1 = **0.3546**
- $L_{KL} = \frac{1}{2}(0.34885 + 0.3546) = \frac{1}{2}(0.70345) = 0.3517 \approx$ **0.352**

Step 10: Total VAE loss

Total VAE Loss = Reconstruction Loss + KL = 9.7111 + 0.352 = 10.0631

Observation: reconstruction dominates (9.71 vs 0.35) because the untrained decoder is very wrong. Training will drive the reconstruction term down while keeping KL small.

Exam-day exactness notes

- Reconstruction loss in the lecture is the *mean* squared error over the 4 features (divide by 4), while the KL term is *summed* over the 2 latent dimensions (½ Σ). Follow whatever the question states; if it says "sum of squared errors" do not divide.
- The slide uses the rounded $\hat{x}$ for the loss. Using unrounded $\hat{x}$ gives a value within about 0.001 of 9.7111.
- Dimension check: x (1×4) W_1 (4×3) → (1×3); a (1×3) W_μ (3×2) → (1×2); z (1×2) W_2 (2×3) → (1×3); a (1×3) W_3 (3×4) → (1×4).

---

7. Formula sheet (one-page revision)

Information theory ($\log_2$ → bits; ln → nats)

- Self-information: $I(x) = -\log_2 P(x)$
- Entropy: $H(P) = -\Sigma P(x) \log_2 P(x)$
- Cross-entropy: $H(P,Q) = -\Sigma P(x) \log_2 Q(x)$
- KL: $D_{KL}(P\|Q) = H(P,Q) - H(P) = \Sigma P(x) \log [P(x)/Q(x)]$ (continuous: $\int$)
- KL ⩾ 0; = 0 iff P = Q; asymmetric. Forward KL $D(P\|Q)$ = mode-covering; reverse KL $D(Q\|P)$ = mode-seeking.

VAE

- Prior: $p(z) = N(0, I)$. Encoder: $q_\varphi(z|x) = N(\mu_\varphi(x), \sigma^2_\varphi(x))$. Decoder: $p_\theta(x|z)$.

- Evidence: $p(x) = \int p(x|z)p(z)dz$ (intractable). Bayes: $p(z|x) = p(x|z)p(z)/p(x)$.
- $\sigma^2 = \exp(\log \sigma^2)$; $\sigma = \sqrt{\sigma^2}$.
- Reparameterization: $z = \mu + \sigma \odot \varepsilon$, $\varepsilon \sim N(0, I)$. $\partial z/\partial \mu = 1$, $\partial z/\partial \sigma = \varepsilon$.
- ELBO = $E_q[\log p_\theta(x|z)] - D_{KL}(q_\varphi(z|x) \| p(z))$; $\log p(x) = \text{ELBO} + D_{KL}(q(z|x) \| p(z|x))$.
- Loss = −ELBO = Reconstruction + KL.
- Reconstruction: MSE (continuous, linear output) or BCE (binary / [0,1], sigmoid output).
- Gaussian KL to N(0,1): $\frac{1}{2} \Sigma (\mu^2 + \sigma^2 - \log \sigma^2 - 1)$.
- Activations: ReLU on hidden layers; **none** on μ and $\log \sigma^2$ layers; output layer linear or sigmoid by data type.
- Parameters: encoder φ, decoder θ. Generation of new data: sample $z \sim N(0, I)$, decode.

Common mistakes to avoid

- Treating the encoder's second output as σ^2 when it is $\log \sigma^2$ (you must exponentiate).
- Using σ^2 where σ is needed in $z = \mu + \sigma\varepsilon$ (take the square root first).
- Applying ReLU to μ or $\log \sigma^2$.
- Saying ε is learned (it is sampled, independent of φ).
- Saying KL is symmetric or a metric.
- Confusing ELBO (maximize) with the loss (minimize, with the sign flipped).
- Writing $N(\mu, \sigma^2)$ with the second argument as the standard deviation (it is the variance).
- Forgetting that the lecture's z-values are samples, so different ε give different z and different $\hat{x}$ for the same x.

---

8. Extra practice numericals (with full solutions)

N1. Self-information. An event has P = 1/16. Find I. *Solution:* $-\log_2(1/16) = \log_2 16 =$ **4 bits**.

N2. Entropy. Find H for [0.5, 0.25, 0.25]. *Solution:* 0.5(1) + 0.25(2) + 0.25(2) = **1.5 bits**. For [0.25, 0.25, 0.25, 0.25]: 4 × 0.25 × 2 = **2 bits** (the maximum, $\log_2 4$).

N3. Cross-entropy as classification loss. True class is Rainy: P = [0, 1]; model Q = [0.875, 0.125]. *Solution:* $H(P,Q) = -\log_2(0.125) =$ **3 bits**. If instead Q = [0.125, 0.875] the loss is $-\log_2(0.875) =$ **0.193**.

N4. KL from entropy. P = [0.5, 0.5], Q = [0.25, 0.75]. Find D(P‖Q). *Solution:* 0.5 $\log_2(0.5/0.25)$ + 0.5 $\log_2(0.5/0.75)$ = 0.5(1) + 0.5(−0.585) = 0.5 − 0.2925 = **0.2075 bits**. Check: H(P,Q) = −0.5$\log_2$0.25 − 0.5$\log_2$0.75 = 1 + 0.2075 = 1.2075; H(P) = 1; KL = 0.2075 ✓

N5. Gaussian KL. μ = [1, −1], σ² = [1, 4]. *Solution (natural log):* j=1: 1 + 1 − 0 − 1 = 1. j=2: 1 + 4 − ln 4 − 1 = 4 − 1.3863 = 2.6137. Sum = 3.6137, half = **1.807**.

N6. Encoder to sample. Use the lecture weights with a = [1.0, 2.0, 0.5]. Find μ, σ, and z for ε = [1, −1].

- μ_1 = 1(0.5) + 2(−0.3) + 0.5(0.1) = −0.05; μ_2 = 1(0.2) + 2(0.4) + 0.5(−0.5) = 0.75
- log σ_1^2 = 1(−0.2) + 2(0.4) + 0.5(0.2) = 0.7; log σ_2^2 = 1(0.3) + 2(−0.1) + 0.5(0.2) = 0.2
- σ² = [e^0.7, e^0.2] = [2.014, 1.221]; σ = [1.419, 1.105]
- z_1 = −0.05 + 1.419(1) = **1.369**; z_2 = 0.75 + 1.105(−1) = **−0.355**
- KL = ½[(0.0025 + 2.014 − 0.7 − 1) + (0.5625 + 1.221 − 0.2 − 1)] = ½(0.3165 + 0.5835) = **≈ 0.450**

N7. ReLU decoder layer. If h = [−0.5, 0.3, 1.2] then a = **[0, 0.3, 1.2]**.

N8. MSE. x = [1, 2, 3], x̂ = [1.5, 2, 2]. *Solution:* (0.25 + 0 + 1)/3 = **0.4167** (sum of squares = 1.25).

N9. Gradient through reparameterization. ε = −0.8, ∂L/∂z = 3. Then ∂L/∂μ = 3 × 1 = **3**, ∂L/∂σ = 3 × (−0.8) = **−2.4**.

N10. Total loss. Reconstruction = 2.5, KL = 0.4 → L = **2.9**, and (conceptually) ELBO = −2.9.

---

9. Practice MCQs (46 questions)

Attempt first, then check the answer key and explanations in Section 10.

Part A: KL divergence and information theory

Q1. D_KL(P ‖ Q) = 0 if and only if: (a) P and Q have the same mean (b) P = Q everywhere (c) Q is Gaussian (d) P is uniform

Q2. Which statement about KL divergence is correct? (a) It is symmetric (b) It can be negative (c) It is not symmetric, so it is not a true distance metric (d) It is defined only for discrete distributions

Q3. Forward KL, D_KL(P‖Q), where P is the true distribution, tends to produce: (a) mode-seeking behaviour (b) mode-covering behaviour (c) a Q that ignores all but the tallest mode (d) a Q with zero variance

Q4. Reverse KL, D_KL(Q‖P), heavily penalizes: (a) Q for missing regions where P has mass (b) Q for placing probability where P has little or no mass (c) P for having multiple modes (d) any difference in the mean

Q5. In forward KL, if P(x) > 0 but Q(x) = 0 for some x, then D_KL(P‖Q) is: (a) 0 (b) 1 (c) negative (d) infinite

Q6. An event has probability 0.125. Its self-information in bits is: (a) 0.125 (b) 1 (c) 3 (d) 8

Q7. The entropy of a distribution over n outcomes is largest when: (a) one outcome has probability 1 (b) the distribution is uniform (c) two outcomes share all the probability (d) the distribution is Gaussian

Q8. The entropy of P = [0.875, 0.125] (log base 2) is approximately: (a) 0 (b) 0.193 (c) 0.544 (d) 1

Q9. True outcome is Sunny, P = [1, 0], and the model predicts Q = [0.125, 0.875]. The cross-entropy is approximately: (a) 0.193 (b) 0.875 (c) 1 (d) 3

Q10. The relationship between KL, cross-entropy and entropy is: (a) D_KL(P‖Q) = H(P) − H(P,Q) (b) D_KL(P‖Q) = H(P,Q) − H(P) (c) D_KL(P‖Q) = H(P,Q) + H(P) (d) D_KL(P‖Q) = H(P)·H(P,Q)

Q11. Self-information uses a logarithm mainly because it: (a) makes values negative (b) turns multiplication of independent probabilities into addition of information (c) removes the need for probabilities (d) makes entropy symmetric

Q12. With limited capacity, a model trained with reverse KL on a cats-and-dogs dataset is most likely to: (a) produce blurry cat-dog hybrids (b) produce sharp images of only one class (c) produce pure noise (d) always cover both classes equally

Q13. Since H(P) is fixed by the data, minimizing cross-entropy H(P,Q) over Q is equivalent to: (a) maximizing entropy (b) minimizing D_KL(P‖Q) (c) maximizing D_KL(P‖Q) (d) making Q uniform

Part B: VAE concepts

Q14. The main reason VAEs were introduced compared with plain autoencoders is: (a) faster training (b) generation of new data (c) fewer parameters (d) supervised labelling

Q15. The VAE described in the lectures is: (a) supervised, using class labels (b) unsupervised, class labels are not used (c) reinforcement-based (d) semi-supervised

Q16. The true posterior $p(z|x)$ is intractable mainly because: (a) the prior is Gaussian (b) $p(x) = \int p(x|z)p(z)dz$ needs integration over all latent z (c) the decoder has too few layers (d) x is continuous

Q17. In a standard VAE the prior $p(z)$ is chosen as: (a) uniform on [0,1] (b) $N(0, I)$ (c) Bernoulli(0.5) (d) learned from the encoder output

Q18. The encoder of a VAE typically outputs: (a) z directly (b) μ and $\log \sigma^2$ (c) $\hat{x}$ (d) only σ

Q19. The encoder outputs $\log \sigma^2$ rather than σ^2 mainly because: (a) log is faster to compute (b) the output can be any real number and $\exp(\cdot)$ guarantees positive variance (c) it makes the mean zero (d) KL requires it to be negative

Q20. Which activation is applied on the μ and $\log \sigma^2$ output layers of the encoder in the lecture? (a) ReLU (b) Sigmoid (c) Tanh (d) None (linear)

Q21. The KL term $D_{KL}(q_\phi(z|x) \| p(z))$ in the VAE loss: (a) improves pixel-level reconstruction (b) keeps the encoder distribution close to the prior (c) removes randomness (d) is the decoder's likelihood

Q22. The ELBO satisfies: (a) ELBO $\geqslant \log p(x)$ (b) ELBO $= \log p(x)$ always (c) ELBO $\leqslant \log p(x)$ (d) ELBO is unrelated to $\log p(x)$

Q23. The gap $\log p(x) - \text{ELBO}$ equals: (a) $D_{KL}(p(z)\|q(z|x))$ (b) $D_{KL}(q(z|x)\|p(z|x))$ (c) the reconstruction loss (d) zero

Q24. The VAE loss is obtained from the ELBO by: (a) adding a constant (b) flipping the sign (maximize → minimize) (c) squaring it (d) taking its exponent

Q25. For continuous data the decoder's reconstruction loss and output activation are typically: (a) BCE with sigmoid (b) MSE with linear (c) BCE with linear (d) MSE with softmax

Q26. For binary data the decoder outputs $\pi_\theta(z)$, which parameterizes: (a) a Gaussian (b) a Poisson distribution (c) a Bernoulli distribution (d) a uniform distribution

Q27. The reparameterization trick is needed because: (a) the decoder is not differentiable (b) sampling $z \sim q(z|x)$ is a random operation that blocks gradient flow to μ and σ (c) the loss has two terms (d) the prior is Gaussian

Q28. In $z = \mu + \sigma \odot \varepsilon$, the derivative $\partial z/\partial \mu$ equals: (a) 0 (b) ε (c) 1 (d) σ

Q29. In $z = \mu + \sigma \odot \varepsilon$, the derivative $\partial z/\partial \sigma$ equals: (a) 0 (b) ε (c) 1 (d) μ

Q30. During backpropagation, ε is: (a) a learned parameter of the encoder (b) sampled from N(0, I) and treated as a constant (c) equal to σ (d) always zero

Q31. For a 1-D Gaussian q = N(0, 1), $D_{KL}(q \| N(0,1))$ is: (a) 0 (b) 1 (c) 0.5 (d) infinite

Q32. $z = \mu + \sigma\varepsilon$ with $\varepsilon \sim N(0,1)$ gives a z distributed as: (a) N(0, 1) (b) $N(\mu, \sigma^2)$ (c) $N(\mu, \sigma)$ (d) $N(\sigma, \mu^2)$

Q33. The correct order of the gradient path in a VAE is: (a) L → φ → z → x̂ (b) L → x̂ → z → (μ, σ) → φ (c) x̂ → L → z → φ (d) z → φ → L → x̂

Q34. In the Gaussian-likelihood case, the decoder output $\hat{x} = \mu_x$ is used in practice because: (a) it is exactly the sample from $p_\theta(x|z)$ (b) in practice the mean of $p_\theta(x|z)$ is taken as the reconstruction (c) σ^2_θ is always 0 (d) sampling is not allowed in neural networks

Part C: Numericals

Q35. In the lecture example the encoder's log $\sigma^2 = [0.732, 0.302]$. The variances σ^2 are approximately: (a) [0.732, 0.302] (b) [2.079, 1.353] (c) [1.442, 1.163] (d) [0.481, 0.739]

Q36. With $\sigma^2 = [2.079, 1.353]$, the standard deviations σ are approximately: (a) [2.079, 1.353] (b) [1.442, 1.163] (c) [4.32, 1.83] (d) [0.732, 0.302]

Q37. With $\mu = [0.043, 0.551]$, $\sigma = [1.442, 1.163]$ and $\varepsilon = [0.5, -0.2]$, z is: (a) [0.764, 0.318] (b) [0.043, 0.551] (c) [0.908, 0.086] (d) [0.721, −0.233]

Q38. For $\varepsilon = [0, 0]$ the sampled z equals: (a) 0 (b) σ (c) μ (d) μ + σ

Q39. For $\varepsilon = [0.6, -0.4]$ with the same μ and σ, z is approximately: (a) [0.764, 0.318] (b) [0.908, 0.086] (c) [0.865, −0.465] (d) [0.043, 0.551]

Q40. In the decoder, h = [0.6538, −0.1738, 0.0938]. After ReLU the hidden activation is: (a) [0.6538, −0.1738, 0.0938] (b) [0.6538, 0, 0.0938] (c) [1, 0, 1] (d) [0.6538, 0.1738, 0.0938]

Q41. In the lecture example the total VAE loss (reconstruction + KL) is: (a) 9.7111 (b) 0.352 (c) 10.0631 (d) 38.8443

Q42. In the lecture example the reconstruction loss is 9.7111 because: (a) it is a mean (÷4) over squared errors of the 4 features, with the 5.1 → 0.103 error dominating (b) it is a sum of 2 latent dimensions (c) it uses BCE (d) x̂ is almost equal to x

Q43. For $\mu = [1, -1]$ and $\log \sigma^2 = [0, 0]$, the KL to N(0, I) is: (a) 0 (b) 1 (c) 2 (d) 0.5

Q44. Entropy ($\log_2$) of P = [0.5, 0.25, 0.25] is: (a) 1 (b) 1.5 (c) 2 (d) 1.585

Q45. An encoder gives $\mu = 1$ and $\sigma^2 = 4$. For $\varepsilon = -0.5$, z is: (a) 0 (b) −1 (c) −0.5 (d) 3

Q46. If $\partial L/\partial z = 4$ and $\varepsilon = 0.25$, then $\partial L/\partial \mu$ and $\partial L/\partial \sigma$ are: (a) 4 and 1 (b) 1 and 4 (c) 4 and 4 (d) 0.25 and 1

---

10. Answer key with explanations

Q1. (b) KL = 0 exactly when the two distributions are identical.

Q2. (c) $D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$ in general, and KL is always ≥ 0, defined for discrete and continuous cases.

Q3. (b) Forward KL punishes Q for missing regions where P has mass, so Q spreads out to cover all of P's support (mode-covering).

Q4. (b) Reverse KL punishes Q for placing mass where $P \approx 0$; Q concentrates on one mode (mode-seeking).

Q5. (d) $\log(P/0) \to \infty$, so the divergence is infinite.

Q6. (c) $-\log_2(0.125) = 3$ bits (since $0.125 = 2^{-3}$).

Q7. (b) Entropy is maximal for the uniform distribution ($\log_2 n$); it is 0 when one outcome has probability 1.

Q8. (c) $0.875 \times 0.193 + 0.125 \times 3 = 0.169 + 0.375 = 0.544$.

Q9. (d) $H(P,Q) = -\log_2 Q(\text{Sunny}) = -\log_2(0.125) = 3$.

Q10. (b) $D_{KL}(P\|Q) = H(P,Q) - H(P)$: the extra bits beyond the entropy.

Q11. (b) $\log(ab) = \log a + \log b$ gives additive information for independent events.

Q12. (b) Reverse KL is mode-seeking: sharp, realistic samples from one mode, other class missed.

Q13. (b) $H(P,Q) = H(P) + D_{KL}(P\|Q)$, and H(P) does not depend on Q.

Q14. (b) The autoencoder learned reconstruction; the VAE is introduced for generation.

Q15. (b) Class labels are not used during training; the model learns the data's pattern.

Q16. (b) The evidence $p(x)$ is an integral over all possible z, intractable for neural decoders.

Q17. (b) The prior is the standard normal $N(0, I)$: simple, easy to sample, gives a continuous structured latent space.

Q18. (b) The encoder outputs the mean vector and the log-variance vector.

Q19. (b) $\log \sigma^2$ is unconstrained; $\sigma^2 = \exp(\log \sigma^2)$ is always positive.

Q20. (d) No standard activation on μ or $\log \sigma^2$; ReLU would wrongly clip negative means.

Q21. (b) The KL term acts as a regularizer pulling $q_\varphi(z|x)$ toward $p(z)$.

Q22. (c) ELBO is a lower bound: ELBO $\leqslant \log p_\theta(x)$.

Q23. (b) $\log p(x) = \text{ELBO} + D_{KL}(q(z|x) \parallel p(z|x))$, so the gap is the KL to the true posterior.

Q24. (b) Networks minimize loss, and $\max(f) \Leftrightarrow \min(-f)$, so $L_{VAE} = -\text{ELBO}$.

Q25. (b) Continuous data: Gaussian likelihood gives MSE with a linear output; binary/[0,1] data: BCE with sigmoid.

Q26. (c) $p_\theta(x|z) = \text{Bernoulli}(\pi_\theta(z))$.

Q27. (b) Randomness in sampling breaks gradient flow back to the encoder outputs; the trick rewrites z as a deterministic function of μ, σ and external noise ε.

Q28. (c) $\partial z/\partial \mu = 1 + 0 = 1$.

Q29. (b) $\partial z/\partial \sigma = 0 + \varepsilon = \varepsilon$.

Q30. (b) ε is not learned, is independent of φ, and is a constant during backprop.

Q31. (a) $\tfrac{1}{2}(0 + 1 - 0 - 1) = 0$.

Q32. (b) Shifting by μ and scaling by σ gives $N(\mu, \sigma^2)$; the second parameter is the variance.

Q33. (b) $L \to \hat{x} \to z \to \mu_\varphi(x), \sigma_\varphi(x) \to \varphi$.

Q34. (b) Theoretically $\hat{x}$ is sampled from $p_\theta(x|z)$; in practice we use its mean μ_x.

Q35. (b) $e^{0.732} \approx 2.079$ and $e^{0.302} \approx 1.353$. [1.442, 1.163] in (c) are the standard deviations, not variances.

Q36. (b) $\sqrt{2.079} = 1.442$, $\sqrt{1.353} = 1.163$.

Q37. (a) $z_1 = 0.043 + 1.442(0.5) = 0.764$; $z_2 = 0.551 + 1.163(-0.2) = 0.318$.

Q38. (c) With $\varepsilon = 0$, $z = \mu$.

Q39. (b) $z_1 = 0.043 + 0.865 = 0.908$; $z_2 = 0.551 - 0.465 = 0.086$.

Q40. (b) ReLU clips only the negative entry: [0.6538, 0, 0.0938].

Q41. (c) 9.7111 + 0.352 = 10.0631.

Q42. (a) Squared errors 24.97, 12.45, 1.43, 0.002 sum to 38.84, divided by 4 gives 9.71; large because the weights are random.

Q43. (b) The formula is ½ Σ (μ² + σ² − log σ² − 1). With log σ² = 0 (so σ² = 1) each dimension reduces to μ² = 1, the sum is 2, and ½ × 2 = **1**.

Q44. (b) 0.5(1) + 0.25(2) + 0.25(2) = 1.5 bits.

Q45. (a) σ = √4 = 2, so z = 1 + 2(−0.5) = 0.

Q46. (a) ∂L/∂μ = 4 × 1 = 4; ∂L/∂σ = 4 × 0.25 = 1.

Final revision checklist

- Can you derive KL = cross-entropy − entropy from scratch?
- Can you explain forward vs reverse KL using the height or cats-and-dogs example?
- Can you list the six distributions/symbols (p_data, p(z), p(z|x), p(x|z), q_φ(z|x), p(x)) and say which one the encoder and decoder represent?
- Can you derive log p(x) = ELBO + KL(q‖p(z|x)) and the closed-form Gaussian KL?
- Can you explain why sampling blocks backprop and how z = μ + σε fixes it, including ∂z/∂μ = 1 and ∂z/∂σ = ε?
- Can you redo the Lec 24 forward pass end-to-end (answer: reconstruction 9.7111, KL 0.352, total 10.0631)?